Lab

Hide and Seek Arena

1 Introduction

Project 1 – Hide and Seek Arena aims to help students understand, implement, and optimize search algorithms through an automated arena that simulates the game of hide-and-seek, where student teams can compete against one another for ranking.

Each student team will develop both of the following agents:

- **Hide agent**: Hide for as long as possible.
- **Seek agent**: Find and catch as quickly as possible.

The agents will be automatically matched in the Arena to compute win rates and rankings.

2 Arena

The Arena is an automated framework, implemented in Python, that serves as the competitive environment for the agents. Given the teams' submissions, the Arena will (1) automatically import each team's agents; (2) run a series of matches between teams; and (3) output the results.

The model of the Arena is illustrated in Figure 1.

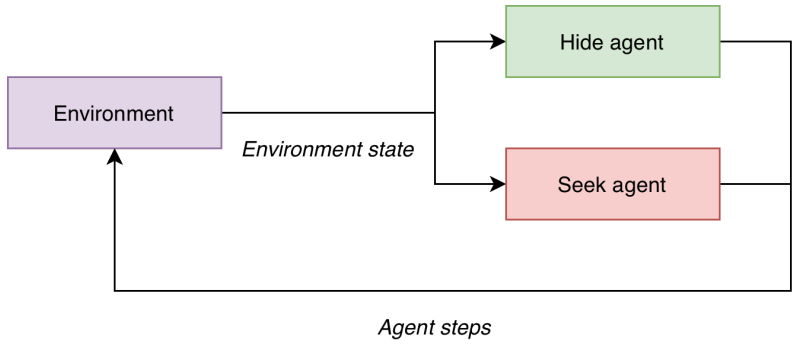


Figure 1: An overview of the Arena model.

Algorithm 1 Agent – Environment Interaction Loop

```
while episode not terminated do
    a_p ← PacmanAgent.step()
    a_g ← GhostAgent.step()
    Environment.step(a_p, a_g)
    Update agent positions
    Check collisions or termination
    s ← observe new environment state
end while
```

At each step in Algorithm 1, the provided information includes:

- A 21×21 map (where 0 indicates a traversable cell and 1 indicates a non-traversable cell).
- The current positions of the two agents.
- The current step count.

Note: Both agents receive the same state information and make their decisions simultaneously. The win conditions are defined as follows:

- The Seek agent wins: if the Seek agent touches the Hide agent (Manhattan distance < 2).
- The Hide agent wins: if the maximum number of allowed steps is reached.

Please note that, the framework is provided to you for reference, evaluting your own agent against the above conditions may require some tweakings.

The source code will be provided to students with the following directory structure:

```
Arena/
├── src/ ..... Framework source code (do NOT modify)
├── submissions/ ..... Student workspace
│   ├── example_student/
│   │   └── agent.py ..... Reference implementation
│   └── student_id/
│       └── agent.py ..... Student team's agent
├── STUDENT_GUIDE.md ..... Student instructions
└── run_game.sh ..... Quick-run script
```

Important note: If both agents move at the same speed, the Seek Agent can never win because it cannot make contact with the Hide Agent. To ensure the fairness, the Seek Agent moves at a speed of 2 cells per step when moving in straight line but cannot move with an L-shaped immediately during a turn.

At each step, a student team's agent must return an action in the following format:

```python
class AgentInterface(ABC):
    @abstractmethod
    def step(self, map_state, my_position, enemy_position, step_number):
        # For Pacman Agent: return a Move or (Move, steps), where:
        # - steps is an int from 1 to the configured max straight-line speed.
        # For Ghost Agent: return a Move (UP, DOWN, LEFT, RIGHT, or STAY).
```

The recommended algorithms are summarized in Table 1.

Table 1: Suggested Algorithms for Seek and Hide Agents

Seek Agent	**Hide Agent**
– Breadth-First Search (BFS)	– BFS: finding the farthest position
– Depth-First Search (DFS)	– Minimax/Expectimax
– A*	– Monte Carlo
– Greedy	– Potential Fields
– etc.	– etc.

3 Tournament

Each team will participate in both roles, Hide and Seek, and will compete against the agents of all other teams. Based on the outcomes, the win rates are computed as:

$$\text{win_rate}_{\text{hide}} = \frac{\text{Hide wins}}{\text{Hide games}} \quad \text{win_rate}_{\text{seek}} = \frac{\text{Seek wins}}{\text{Seek games}}$$

The results are recorded in the following example table.

Table 2: Match Results Between Teams (Hide vs. Seek)

Hide \ Seek	**Team 1**	**Team 2**	**Team 3**	**Team 4**
Team 1	–	0	1	1
Team 2	0	–	1	0
Team 3	1	0	–	1
Team 4	0	0	1	–

Here, a value of 0 denotes a Hide win, while a value of 1 denotes a Seek win.

Based on Table 2, an example of the win rates for Team 1 is as follows:

- As the Hide agent, Team 1 wins 1 out of 3 matches, resulting in $\text{win_rate}_{\text{hide}} = 33.3\%$.
- As the Seek agent, Team 1 wins 2 out of 3 matches, resulting in $\text{win_rate}_{\text{seek}} = 66.6\%$.

Note that, besides raw winning result, we also record your agent's average steps required to end each match. For Pacman, you should also minimize this value while for Ghost, you should also maximize this value. In order to tie-breaking the teams with similar winning rate, we evaluate their differences between Pacman's and Ghost's average steps where one with lower difference ranks higher.

The scoring criteria for each team is defined as follows:

Criterion	**Score**
Algorithm implementation completeness	3
Ranking in the initial submission (see timeline in Section 4)	Up to 3
Ranking in the optimized submission (see timeline in Section 4)	Up to 4

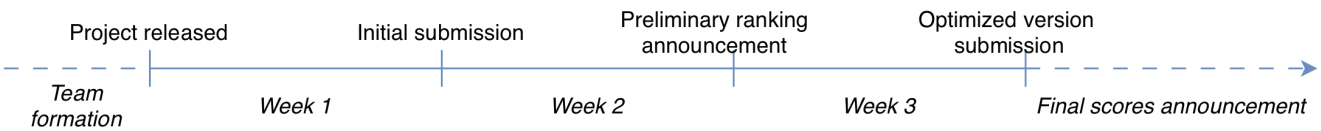
4 Submission

Each team must submit a single `.zip` file to Moodle containing the following directory structure where `group_id` denotes a single integer for your group's ID only:

```
group_id/
├── agent.py
└── etc.
```

The submitted agent is allowed to import other files located within the same submission directory.
Note: Late submissions will **not** be accepted under any circumstances.

The submission timeline is specified as follows:



5 Notices

Please pay attention to the following notices:

- This is a **GROUP** assignment. Each group has 3 - 4 members.

  Groups with fewer or more members than the specified limit require lab instructor approval.

- This project has two submission phases. Only teams that submitted in the first phase are allowed to submit in the second phase.

- Student teams must ensure that their agent returns an action within a maximum of **1 second**. The built-in `"time"` library may be used together with all built-in libraries bundled with Python 3. External libraries available for your team includes: numpy, pandas, scipy, and gurobi.

- AI tools are **NOT restricted**; however, students should use them wisely. Lab instructors have the right to conduct additional oral interviews to assess their knowledge of the project.

- Any form of plagiarism, dishonesty, or misconduct will result in a grade of zero for the course.

The end.